Taehyun Kim

Frontend Engineer taehyun_fe@naver.com · please reach out by email · github.com/qlemql

PROFESSIONAL SUMMARY

I investigate user problems through code and production data, then take on the work needed to resolve them. I have owned frontend development for booking and payment services and worked across teams to fix issues between order and advertising systems. I turn repeated review and verification work into shared AI tools and real-device tests.

WORK EXPERIENCE

T-order

Frontend Engineer Mar 2026 – Present

Full-time | Ad Platform team (2 FE) → Frontend Chapter (5)

Built an owner-facing React webview and integrated a Vue Order app with a React ad module. After moving from Ad Platform to Order, continued working across both teams to resolve production issues and build developer tools.

1. T-order AI store-linking webview — new, 0→1

CONTEXT

Extended store search and sales-report requests from a KakaoTalk chatbot into a webview, first mapping requests and authentication across multiple backends.

WHAT I DID

- Used AI to analyze backend repositories and agreed the integration design with backend engineers: one server endpoint for the webview, server-held tokens, and a session ID passed by the client.
- Built the screens and API integration with React, TypeScript, and TanStack Query; reused MSW responses in Storybook and tests to verify login and terms flows.

RESULT

Shipped store search, authentication, report requests, and campaign entry flows.

2. AI workflow automation and shared developer tools

CONTEXT

Turned recurring review, release, and documentation work into tools the team could reuse.

WHAT I DID

- Distributed Claude Code review, release, and documentation skills as a shared FE plugin. Found and fixed a duplicate-transaction path in a teammate's prepaid-payment PR.
- Built an agent on an NHN server for release plans, sprint retrospectives, Sentry investigation, and code checks, connecting results to relevant Slack threads.
- Built a personal macOS app connecting documents, tickets, PRs, and Claude sessions. Verified a trial workflow from estimates and tickets to a draft PR with human approvals; currently using and improving it myself.

RESULT

Used the release skill for two regular releases and one hotfix, adding repository-specific settings and checks against deployed code.

3. Resolving a four-month repeated-query investigation

CONTEXT

Teams investigated get_cart_list call sites in their own code, but the system-wide repeating cycle went unexplained for over four months.

WHAT I DID

- With AI assistance, first found a roughly 41-second request interval in OpenSearch. Using experience in both systems, traced the code and store settings to a repeating 30-second idle timer and 10-second default image.
- Discussed existing policy options with Ad Platform and proposed continuous news playback when no ads were scheduled. Compared matching-Sunday logs for two stores after that team applied the change.

RESULT

Queries across the two stores fell from 40,148 to 3,724 (90.7%), while order submissions held steady or increased. The cycle was broken without a code change.

4. Order-ad integration and payment-aware display control

CONTEXT

Showing the iframe before checking ad-module readiness caused touch-blocking complaints: an empty transparent iframe could cover the visible menu.

WHAT I DID

- Kept the iframe pre-created but showed it only after the readiness ACK agreed with Ad Platform. Handled missing and late responses and fixed an earlier ad's close message clearing the next ad's pending ACK.
- Fixed reactive-object postMessage failures in the shared sender and consolidated display conditions to block simulcast during payment, cancellation, and POS acceptance waits.

RESULT

Owned the Order-side changes and shipped them after regression testing and QA, addressing ad-integration failures that interrupted ordering and payment.

Device testing and production improvements

- Used AI to reduce the burden of writing tests and configuring ADB, building ADB/GeckoView E2E tools for carts, order dialogs, and ad iframes, with setup guides for other teams.
- Removed about 17,500 unused lines across 142 files after checking theme usage with the backend team. Documented the cause, impact, and response options for delayed room restoration after socket reconnection.

Riad Corporation (Ria)

Frontend Engineer → Product Engineer Mar 2024 – Sep 2025 (1 yr 6 mos)

Full-time | Tech team · Owned customer-web frontend development

Extended the B2B group-travel quoting platform Ria into a B2C OTA (online travel agency) product. Built and led the entire frontend.

1. Mobile booking and payment expansion

CONTEXT

Extend the existing B2B service so individual (B2C) customers can use it on equal terms. Goal: instant reservation and payment for both audiences to widen the customer base.

WHAT I DID

- Domestic Toss payments were ready, but international payments awaited bank approval. Selected and integrated Stripe with the CTO in three days; after the CTO's initial implementation, owned payment screens, SDK changes, and subsequent improvements.
- Managed booking inputs with React Hook Form and separated payment options, provider integration, and API calls into hooks. Connected mobile input restoration after redirects and desktop responses to server payment verification and booking confirmation.

RESULT

Launched mobile booking and payment and used Datadog to investigate user errors.

Other key work

- Shipped 3 social logins in 8 days — built the OAuth flow end to end, from frontend to the Go backend's callback/token-exchange endpoints → signup 0.93% → 3.00% (3.2×), 76% via social.
- Queried actual database metrics and planned daily sprints with the PO and PD through development and deployment. Over two months, quote views rose from 58% to 90% and guest quote requests from 53% to 62%.
- In the major Smart Hotel Finder project, replaced 12 input fields with three category choices and added recommendations, reducing quote preparation from ten minutes to under three.
- MVP — pnpm workspace monorepo; streamed AI responses over SSE with auto-reconnect, and unified 5+ response types under a Strategy Pattern.
- Cut build time from 4 min to 1 min (−75%) — measurement showed restoring the CI cache took longer than reinstalling dependencies outright, so removed caching entirely. Dropping the assumption that caching always pays was the whole fix.
- Standardization — code splitting and lazy loading, a standardized TypeScript type system, i18n (ko/en), and GA4/Clarity/Datadog integration.

Lighthouse

Frontend Engineer Dec 2021 - Jan 2023 / Mar 2023 - Jan 2024 (rehired)

Full-time | Engineering (led 2 junior engineers)

Operated and developed the Dailybook web frontend.

Bundle −17% 1.78 → 1.47MB **CI workflow −2min** 7min → 5min

1. FamilyCare — partnership with KidsNote (Mar 2023 – Jan 2024)

CONTEXT

Working in one codebase with an external company (KidsNote) exposed that the team's familiar norms had never been written down.

WHAT I DID

- Built a long-term-care operations system with KidsNote (external) — as team lead, owned task allocation, scheduling, PR-review practices, and tech specs.
- Led the JS → TypeScript migration — standardized component types with utility types and generics.
- GitHub Actions optimization; separated concerns and introduced the Compound Component pattern.
- Canvas e-signature, role-based menu permissions, third-party CMS integration (React, TypeScript, Canvas).

RESULT

Type errors now fail at compile time instead of runtime — CI cut from 7 to 5 minutes.

2. Dailybook — daily care-log service (May 2022 – Jan 2024)

CONTEXT

A daily care-log (guardian–center communication) and ERP service for day-care centers serving people with disabilities — duplicated components, state-management overhead, and slow image loading piled up as scope grew.

WHAT I DID

- Incrementally migrated server state from Redux Saga to React Query (5-min cache + request dedup).
- Built the component system on Atomic Design and styled-components, documented it in Storybook, and applied code splitting and lazy loading; as FE lead, drove conventions and review culture.

RESULT

Server-state handling consolidated from 3 files to 1 hook — bundle down from 1.78 to 1.47MB (−17%).

Other key work

- Jangji-Helpcall — solo-built a responsive web app matching families to burial sites and shipped it to production (Naver Maps, React Query).
- TodayCare — replaced a daily care-log MPA with an SPA in one cutover, introduced Sentry to get error tracking in place, and optimized Webpack (Atomic Design, TypeScript).
- Returned as a mentor to the bootcamp I had graduated from a year earlier, coaching a React cohort alongside my full-time role (Nov–Dec 2022).

SKILLS

Frontend Core React 18/19 · TypeScript · Vue 3 · React Native / Expo · Zustand · Next.js

Data & Testing TanStack Query · SSE · Vitest · MSW

Integrations OAuth · Stripe / Toss · Google / Naver Maps · Sentry / Datadog

Build & Styling Tailwind · Vite / Webpack · pnpm workspace · GitHub Actions · Storybook

AI Collaboration Claude Code · MCP · Custom Skill / Hook · Claude API

Process & Practice PR review · Tech specs · Experimentation · Onboarding docs

SIDE PROJECTS

Ssak — Habit Tracker

App Store (https://apps.apple.com/app/id6762334017)
Play Store (https://play.google.com/store/apps/details?id=com.qlemql.minimalhabittracker)
GitHub (https://github.com/qlemql/minimal-habit-tracker)

- Released and maintain a habit-tracking app on iOS and Android. Built local storage, notifications, widgets, and backup/restore, and promote the app on Instagram.

Fox Walk

App Store (https://apps.apple.com/app/id6822642908)
Play Store (https://play.google.com/store/apps/details?id=com.foxwalk.app)

- Released an iOS and Android walking app where a fox grows as users walk. Connected step counts, weather, and widgets, then fixed repeated popups and notification and storage failures after release.

EDUCATION

Hanghae99 — Software Engineering Bootcamp Sparta Coding Club (Korean coding bootcamp) | Jun–Sep 2021

Catholic Kwandong University BA in Chinese Studies | Mar 2013 - Aug 2019